

Problem Statement

The objective of this project is to design, implement, and analyze a digital communication system that incorporates error-detection techniques. The problem focuses on simulating the transmission of binary data through a communication channel, intentionally introducing errors, and evaluating how effectively the system detects those errors. This project specifically limits its scope to the use of **parity checks** and **basic block** coding methods for error detection,

Implementation Details

This project is implemented in C# using the Avalonia UI framework, following object-oriented programming (OOP) principles. During the development process, different data representations for binary values were considered. Initially, variables were defined using the *uint* type; however, this approach resulted in the loss of leading zeros, which are significant in binary data representation. To address this issue, a *string*-based representation was explored, as it preserves the exact bit structure. Nevertheless, it was ultimately determined that performing bitwise operations is more efficient and appropriate with the *uint* type. Therefore, the implementation primarily utilizes *uint* for computation, while handling representation concerns as needed. The simulation is designed to operate on dataword sizes ranging from 1 to 16 bits, allowing flexible testing across different input lengths.

Scenario Testing

This section introduces testing screenshots of the simulation.

Comparative Analysis

Parity check (even and odd) and basic block coding methods exhibit similar performance in error detection. Both techniques are highly effective at identifying single-bit errors in transmitted data. However, their reliability decreases in the presence of multiple-bit errors, where they may fail to detect corruption or incorrectly interpret the received codeword, potentially leading to erroneous dataword extraction. Despite these limitations, both methods perform accurately and without issues when the transmission channel is error-free.

(a) Even Parity Check No Error

(b) Even Parity Check Single Bit Error

(c) Even Parity Check Burst Error

Figure 1: Parity Check Tests

(a) Block Coding No Error

(b) Block Coding Single Bit Error

(c) Block Coding Burst Error

Figure 2: Block Coding Tests

Conclusion

While creating this simulation I realized that even if implementation of Parity Check is easier than Block Coding it only works odd number corruption if it is even check type, if odd check type then the checker can only find even number corruption but I should mention that Parity Check method only adds 1 bit to the data in this perspective it more convenient to use Parity Check for single-bit corruption but I also say that probability of occurring single-bit corruption is uncommon. On the other hand Block Coding method was hard to implement also it adds more redundant bits to the data, even if it increase the size of the data will be sent it improves reliability. In further

implementations Block Coding method not only detects error also may correct it.